

Graph-Guided Long-Horizon Robotic Assembly via Vision-Language Reasoning and Logic-Geometric Programming

Anonymous Authors

Abstract—

I. INTRODUCTION

Long-horizon multi-part robotic assembly is a fundamental capability for intelligent manufacturing and embodied robotics, with broad applications in both industrial production and daily household environments [1], [2]. Unlike single-step manipulation, these tasks require the robot to accomplish a sequence of interdependent assembly operations while satisfying task dependencies and physical constraints [3]. Automating such processes could greatly expand the practical deployment of robotic assembly systems by reducing manual programming effort and improving adaptability to diverse tasks. However, achieving autonomous long-horizon assembly remains challenging, as the robot must accurately infer the assembly objective, generate a logically consistent task plan, and produce physically feasible motions for each assembly step [4].

Classical approaches to long-horizon robotic assembly are primarily built on model-based Task and Motion Planning (TAMP) [5], [6], which typically employs symbolic planners, such as Planning Domain Definition Language (PDDL) [7], [8], to generate task plans, followed by motion planners to compute feasible trajectories for each action. A representative TAMP approach is Logic-Geometric Programming (LGP) [9], which integrates task planning and motion planning into a unified optimization framework, allowing motion feasibility to be considered during task planning rather than as a separate step. By explicitly modeling task semantics and constraints, these methods produce interpretable and verifiable planning results [10]. However, they heavily rely on manually specified task definitions, object models, and action rules, requiring substantial engineering effort and domain expertise. Consequently, extending these methods to new objects or assembly objectives often needs manual redesign of symbolic models and planning rules, limiting their scalability and adaptability to diverse real-world scenarios.

Recently, Large Language Models (LLMs) have been introduced to robotic assembly to improve high-level task understanding and planning [11], [12]. By leveraging their reasoning capabilities, LLMs are able to directly interpret human instructions, infer assembly intent, and generate task plans without manually defined symbolic specifications [13]. This capability greatly reduces engineering effort and improves adaptability to diverse tasks, thereby enabling more flexible robotic assembly systems in changing environments. However, the generated task plans are not guaranteed to be logically correct or physically executable, as LLMs lack

reliable reasoning over kinematic constraints and motion feasibility [14]. Another practical limitation is that most existing LLM-based approaches assume natural language instructions as input [15], whereas many real-world assembly tasks, particularly in industrial settings, are specified by target assembly images rather than textual descriptions.

ZZ: to change and highlight contribution/novelty In this work, we aim to realize autonomous long-horizon robotic assembly directly from a target assembly image while ensuring safe and reliable motion execution. To this end, we propose a unified framework that combines the visual reasoning capability of Vision-Language Models (VLMs) with the optimization-based guarantees of LGP. Given a target assembly image and the current scene, the framework infers the assembly goal, identifies the required object components, determines an appropriate assembly sequence, and generates executable robot motions that satisfy kinematic and dynamic constraints (see Fig. 1). Specifically, it consists of three key components: (1) *VLM-Based Task Understanding*: the VLM first interprets the target assembly image to construct an assembly graph, from which assembly action plans are derived through graph decomposition and clustering; (2) *Reachability-Manipulability-Aware Scene Understanding*: the robot then perceives the workspace, identifies the corresponding manipulatable objects, and prioritizes them according to reachability and manipulability; (3) *LGP-Based Motion Execution*: finally, the derived task plan and scene information are integrated into an LGP to produce collision-free and physically feasible motion trajectories for completing the assembly task. Note that, the computational complexity of LGP increases rapidly with the number of objects, making its direct usage in long-horizon assembly problems impractical. To address this issue, we decompose the overall assembly process into a sequence of short-horizon subtasks during the VLM-based task understanding, and introduce an active-constraint LGP formulation to further improve computational efficiency.

ZZ: add experiments and results descriptions We evaluate our methods on... The results show that...

ZZ: add contributions Our main contributions are three-fold:

- A constrained VLM interface for extracting an assembly representation from a target image;
- A formal, precedence-preserving decomposition that produces solver-bounded LGP subproblems;
- An end-to-end evaluation of graph quality, planning scalability, and physical execution.

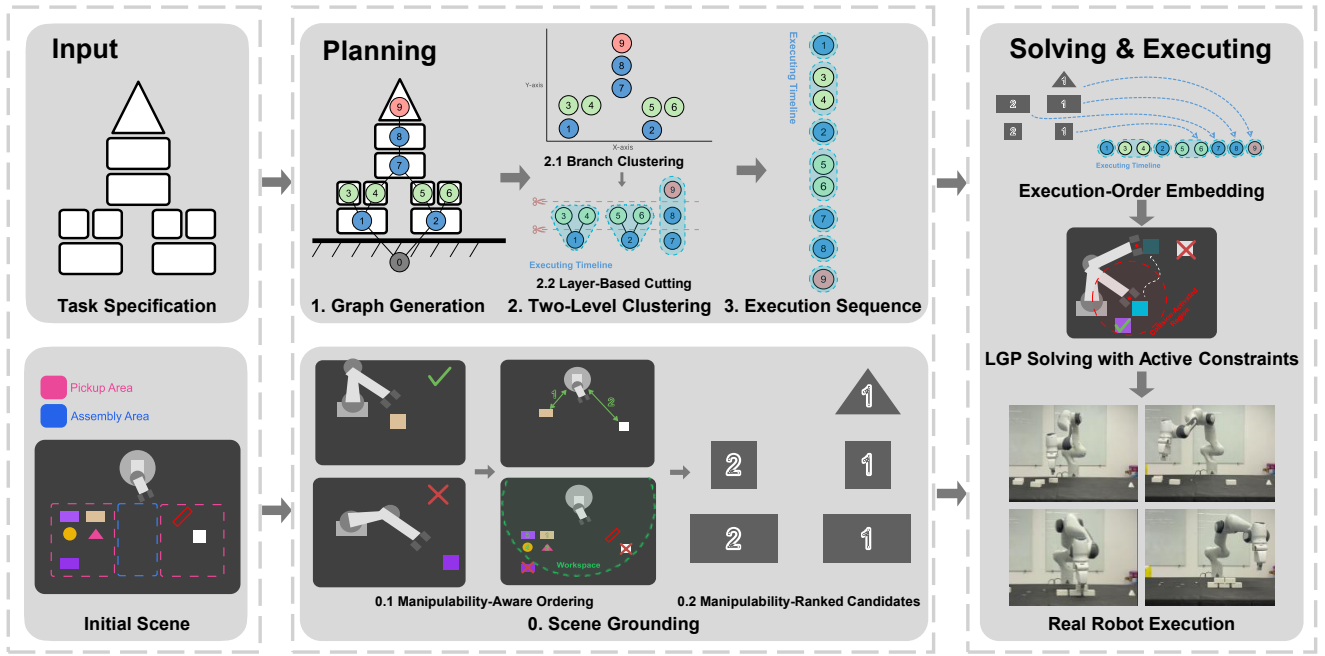


Fig. 1. The overall workflow of the proposed VLM-LGP framework. ZZ: change this figure

II. RELATED WORK

ZZ: check content later

A. Assembly Sequence Planning and Structural Representations

Assembly Sequence Planning (ASP) is typically defined as the process of transforming the final target structure into a series of executable component assembly operations. Classic ASP research typically employs graph structures to represent feasible assembly sequences, constraint relationships, and sub-assembly states. Typical methods include precedence graphs, contact graphs, support graphs, and AND/OR graphs. There are also graph decomposition methods for construction and stacking tasks [16]. The priority graph is typically used to represent sequential constraints between different assembly operations [17]. The contact graph and connection graph are used to describe physical contact, connection, or interference relationships between parts [18]. The support graph further considers factors such as gravity, load-bearing capacity, and the stability of intermediate structures. The AND/OR graph decomposes assembly tasks into multiple intermediate structures, thereby reducing search complexity. For building construction and block-stacking tasks, the graph decomposition method is often combined with stability constraints to ensure that the intermediate structures generated at each step are physically feasible [19].

In recent years, the input formats for assembly tasks have expanded from traditional CAD models to include images, instruction manuals, and multi-modal instructions [20]. However, most methods still rely on known geometric models, predefined part relationships, manually designed support relationships, or assembly instructions that already contain step information. In other words, existing methods

typically assume that assembly relationships are already given, rather than deriving usable structures directly from visual information. Unlike these methods, the system proposed in this paper extracts a structured representation from target images for LGP solving. LGP transforms the target assembly state in the image into an input containing symbolic relationships and geometric constraints. This bridges the gap between visual task descriptions and executable autonomous assembly planning.

B. Task and Motion Planning for Long-Horizon Assembly

Long-horizon assembly tasks can typically be modeled as Task and Motion Planning (TAMP) problems, in which discrete assembly decisions and continuous robot motion must be considered together. In integrated TAMP frameworks such as LGP, symbolic task selection and geometric motion constraints are solved through a unified optimization process. LGP is a typical TAMP formulation method that combines discrete logic programming with continuous trajectory optimization [21]. Since the feasibility of assembly operations depends not only on logical order but is also influenced by constraints such as reachability, stability, and contact relationships, LGP is suitable for describing complex assembly tasks. However, when assembly tasks involve multiple objects and long sequences of operations, directly solving the LGP problem leads to a rapid expansion of the search space.

To improve the scalability of long-horizon LGP, existing research typically employs methods such as hierarchical LGP [22], receding-horizon LGP [23], dynamic LGP [24], heuristic search [25]. These methods can reduce the scale of a single solution. However, fixed-horizon may overlook object dependencies and support relationships within the assembly. And the receding-horizon method may lack global

structural information and require frequent re-planning. The decomposition method proposed in this paper is not based on the fixed-horizon but rather divides the task according to the assembly structure inferred from the target image. By extracting object relationships, support dependencies, and sub-assemblies, this structural information is defined as individual LGP subproblems. The method proposed in this paper preserves the global assembly intent while keeping the subproblems simple.

C. Foundation Models in Robotic Assembly

Foundational models have been widely adopted in robotic manipulation and assembly tasks to enhance task understanding and long-term planning capabilities [26]. Some studies utilize Large Language Models (LLM) to generate high-level task plans based on natural language instructions [27]. Other studies utilize Vision-Language Models (VLM) to identify objects in images, infer spatial relationships, and generate assembly steps [28], [29], [30], [31]. Furthermore, Vision-Language-Action (VLA) models attempt to directly map vision observations and language instructions to robotic actions, thereby enabling end-to-end manipulation control [32], [33], [34]. While these methods enhance task understanding and action generation, they lack an explicit representation of geometric feasibility, contact constraints, support dependencies, and stability conditions in long-term assembly tasks.

To improve reliability, many studies adopt modular systems that combine LLM/VLM with classical planners, linking perception and planning through scene graphs, object relationships, and constraints [35]. Compared to free-form text, this type of structured output is better suited for robotic execution. However, most existing methods are designed for general scene understanding and are not specifically optimized for tasks and motion planners based on LGP. The VLM proposed in this paper outputs a constrained representation of assembly relationships, extracting information directly from target images. And converts it into input for LGP. Therefore, this paper does not directly generate actions using the VLM. This paper uses the VLM to establish an interface between vision target descriptions and LGP long-time-domain assembly planning.

III. PROBLEM FORMULATION

We consider the problem of accomplishing long-horizon robotic assembly directly from a target image, without relying on manually specified symbolic task descriptions. We generally assume a single target image $\mathcal{I}$ in which all components are clearly visible. When occlusions prevent the complete assembly structure from being observed, a sequence of target images is provided to collectively specify the desired configuration (see Fig. 2 for an example). For simplicity, we use $\mathcal{I}$ to denote the visual input in both cases. In addition, the current scene is provided as $\mathcal{S} = \{o_1, \dots, o_N\}$, where each o_i denotes a concrete object instance characterized by its type $c(o_i)$, pose, and geometric attributes. Unlike many existing approaches that assume only the required objects are

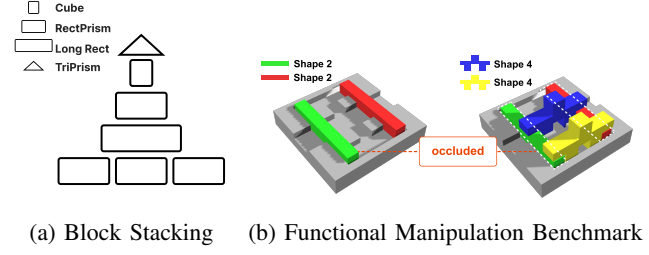


Fig. 2. Examples of target images used in our experiments. (a) For the Block Stacking (BS) task, where all components are clearly visible, a single target image is provided, with the legend indicating the object type $c(v_i)$. The assembly relation set is defined as $\mathcal{R} = \{\text{left_on}, \text{right_on}, \text{center_on}\}$. (b) For the Functional Manipulation Benchmark (FMB) [36] task, where occlusions exist, a sequence of target images is provided to collectively specify the assembly structure, with $\mathcal{R} = \{\text{left_in}, \text{right_in}, \text{center_in}\}$. **ZZ: how $\mathcal{R}$ is defined for FMB? also change (a) so all legends indicated shape are present in the structure, (b) change color to the same for all blocks**

present, we consider a more practical setting where multiple object instances of the same type as well as irrelevant objects may coexist in the scene. Therefore, the components inferred from $\mathcal{I}$ should be associated with suitable object instances in $\mathcal{S}$ before motion execution.

Based on these inputs, image-based autonomous assembly requires addressing the following key problems:

- First, the target assembly image $\mathcal{I}$ needs to be transformed into a structured representation that captures the desired configuration and enables action sequence generation. In this work, we represent this structure using an assembly graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, obtained as
- $$\mathcal{G} = f_G(\mathcal{I}), \quad (1)$$
- where f_G denotes the image-to-graph conversion. Each node $v_i \in \mathcal{V}$ represents a required assembly component with object type $c(v_i)$, while each directed edge $e_{i,j} \in \mathcal{E}$ indicates that object v_i is connected to its supporting object v_j through a spatial relationship $r(e_{i,j}) \in \mathcal{R}$ selected from a predefined set $\mathcal{R}$ of assembly relations.
- Second, a high-level task plan $\mathcal{P}$ needs to be derived from the assembly graph $\mathcal{G}$ to determine the required assembly actions and their execution order.
 - Lastly, the task plan $\mathcal{P}$ needs to be grounded in the current scene $\mathcal{S}$ and translated into executable robot motions. This involves selecting the appropriate concrete object instance for each assembly action and generating collision-free, physically feasible motion trajectories that satisfy the robot's kinematic and dynamic constraints.

IV. METHOD

As illustrated in Fig. 1, the framework consists of three components. First, the target assembly image is converted into an assembly graph, which is then decomposed into a sequence of short-horizon assembly task plans (Sec. IV-A). Second, objects in the current scene are prioritized according to reachability and manipulability for facilitating motion execution (Sec. IV-B). Finally, the task plans and scene information are integrated into a LGP formulation to produce feasible robot motions for assembly execution (Sec. IV-C).

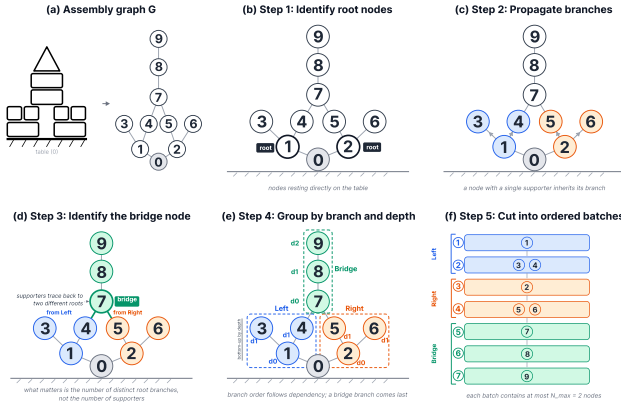


Fig. 3. Example of graph decomposition with $N_{\max} = 2$. ZZ: change this figure

A. Assembly Graph Generation and Decomposition

Leveraging its strong visual reasoning capability, we employ a VLM to perform the image-to-graph conversion f_G . The resulting graph $\mathcal{G}$ is further decomposed to derive a high-level assembly task plan $\mathcal{P}$. Unlike many existing approaches that directly use VLMs to generate action sequences, our method restricts the VLM to extracting the assembly structure, thereby reducing the risk of unreliable planning outputs.

1) *VLM for Assembly Graph Generation*: The VLM-based conversion f_G is performed in a zero-shot manner. To improve generation accuracy, the prompt guides the VLM to focus on identifying individual assembly edges rather than directly predicting the complete graph. Starting from the root nodes, i.e., the base objects of the assembly, connections are progressively inferred layer by layer toward the upper structure. During generation, each inferred edge is also checked against visual evidence in the target image, including whether the lower object can support the upper one and whether their relative positions are consistent with the inferred relation. These checks help improve the quality of the generated graph.

To further enhance correctness, we also introduce a rule-based validator to verify the generated assembly graph $\mathcal{G}$ through an iterative self-refinement process. Specifically, the graph is evaluated against a set of predefined structural and syntactic criteria, such as whether all nodes are connected by valid edges, whether isolated nodes exist, and whether the encoded relations $r(e_{i,j})$ belong to $\mathcal{R}$. If any violation is detected, the corresponding validation feedback is provided to the VLM for regenerating the graph. This validation and refinement procedure is repeated until all criteria are satisfied or a maximum number of iterations is reached. The final validated assembly graph $\mathcal{G}$ then serves as the basis for subsequent task planning.

ZZ: an example of the prompt might be needed here or in the result part

2) *Graph Decomposition for Task Planning*: We then derive the high-level task plan $\mathcal{P}$ by decomposing the assembly graph $\mathcal{G}$. Note that, treating each node as an individual planning step would lead to overly fragmented and inefficient execution, whereas directly passing the entire

graph $\mathcal{G}$ to LGP results in a large optimization problem that becomes computationally intractable. Hence, to balance planning efficiency and computational complexity, we partition the assembly graph $\mathcal{G}$ into a sequence of smaller subgraphs, with each containing at most $N_{\max}$ nodes.

We first perform *branch grouping* to coarsely decompose $\mathcal{G}$ according to assembly dependencies. Specifically, for each node $v \in \mathcal{V}$, we define its set of supporting predecessors as

$$\text{pre}(v) = \{u \in \mathcal{V} \mid \exists e_{v,u} \in \mathcal{E}\}. \quad (2)$$

A node with no supporting object, i.e., $\text{pre}(v) = \emptyset$, corresponds to an object placed on the table and is identified as a *root node*. Each root node serves as the starting point of an individual branch, and all nodes that can be traced back to the same root through a unique supporting-predecessor chain are initially grouped into that branch (see Fig. 3 for an example). Formally, for a non-root node with $|\text{pre}(v)| = 1$, its branch assignment follows that of its unique predecessor. In contrast, a node satisfying $|\text{pre}(v)| > 1$ is defined as a *bridge node*, since its assembly requires all supporting branches to be completed beforehand. Such a node therefore terminates its incoming branches and serves as the starting point of a new branch containing the nodes assembled above it. In this way, root and bridge nodes define the structural boundaries that coarsely partition the assembly graph $\mathcal{G}$ into branches while preserving the required assembly dependencies.

We then perform *branch ordering and partitioning* to obtain the subgraphs used for high-level task planning. The branches are first ordered from bottom to top according to their dependencies, such that a branch starting from a bridge node is considered only after all of its supporting branches have been completed. Branches at the same dependency level are deterministically ordered using predefined spatial rules, e.g., from left to right, to maintain a consistent planning sequence. Within each branch, nodes are grouped by their graph depth measured from the corresponding root or bridge node, such that nodes at the same depth belong to the same group. These groups are then processed in ascending order to ensure that lower-level objects are assembled first. A group containing no more than $N_{\max}$ nodes directly forms a subgraph; otherwise, it is further divided using the same spatial rules into multiple ordered subgraphs, each containing at most $N_{\max}$ nodes. The overall decomposition is given as

$$\mathcal{G} \rightarrow \{\mathcal{G}_1, \mathcal{G}_2, \dots, \mathcal{G}_K\}, \quad (3)$$

where $\mathcal{G}_k = (\mathcal{V}_k, \mathcal{E}_k)$ denotes the k -th subgraph and satisfies $|\mathcal{V}_k| \leq N_{\max}$. Each subgraph $\mathcal{G}_k$ defines a short-horizon task plan $\mathcal{P}_k$, and the complete high-level plan $\mathcal{P}$ is given by

$$\mathcal{P} = (\mathcal{P}_1, \mathcal{P}_2, \dots, \mathcal{P}_K), \quad \mathcal{P}_k \equiv \mathcal{G}_k, \quad (4)$$

which preserves the required assembly dependencies while limiting the size of subsequent LGP planning problems.

B. Reachability-Manipulability-Aware Scene Understanding

Before motion planning, the robot needs to identify suitable concrete object instances corresponding to the components specified in the assembly graph $\mathcal{G}$. To this end, we

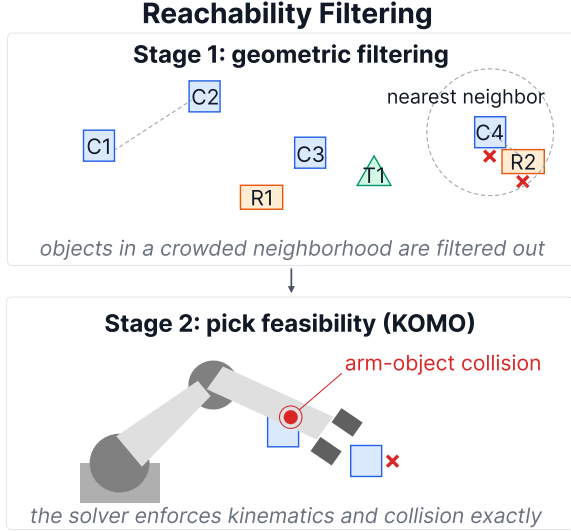


Fig. 4. Reachability filtering. ZZ: change this figure

evaluate each object in the current scene $\mathcal{S} = \{o_1, \dots, o_N\}$ based on reachability and manipulability. Reachability analysis excludes candidates that cannot be feasibly accessed by the robot, while manipulability analysis ranks the remaining ones from high to low according to the ease of manipulation.

1) *Reachability Filtering*: To efficiently exclude objects that are difficult for the robot to access, such as those closely surrounded by other objects, we first perform a coarse *geometric filtering* based on the spatial distribution of objects in the current scene $\mathcal{S}$ (see Fig. 4). For this, we define a geometric accessibility score $\rho(o_i)$ as

$$\rho(o_i) = \rho_{\text{spa}}(o_i) + \alpha \rho_{\text{clear}}(o_i), \quad (5)$$

$$\rho_{\text{spa}}(o_i) = 1 - \frac{1}{N-1} \sum_{j=1, j \neq i}^N \exp\left(-\frac{\|p_i - p_j\|^2}{2\sigma^2}\right), \quad (6)$$

$$\rho_{\text{clear}}(o_i) = \frac{\min(\min_{j \neq i} \|p_i - p_j\|, d_{\max})}{d_{\max}}, \quad (7)$$

where p_i and p_j denote the positions of objects o_i and o_j , respectively. σ controls the neighborhood scale, $d_{\max}$ is a predefined saturation distance, and α is a weighting factor. The local sparsity term ρ_{spa} characterizes the density of neighboring objects, while the clearance term ρ_{clear} measures the distance to the nearest neighbor. Higher values of these terms indicate a less crowded neighborhood and greater surrounding free space, respectively.

We then perform a detailed reachability analysis for objects satisfying $\rho(o_i) \geq \tau_\rho$, where τ_ρ is a predefined threshold. Specifically, for each retained object o_i , we determine whether the robot can grasp it through a feasible collision-free motion. This is formulated as a short-horizon motion optimization problem using the robot kinematic model and the k-Order Markov Optimization (KOMO) [37]. Let $q_{0:T}$ denote the robot joint trajectory and x_i^g a desired grasp pose

of o_i . The reachability problem is given by

$$q_{0:T}^* = \arg \min_{q_{0:T}} \sum_{t=1}^T \|q_t - q_{t-1}\|^2, \quad (8)$$

$$\text{s.t. } \phi_{\text{grasp}}(q_T, x_i^g) = \mathbf{0}, \quad (9)$$

$$q_{\min} \leq q_t \leq q_{\max}, \quad t = 0, \dots, T, \quad (10)$$

$$d_{\text{col}}(q_t, \mathcal{S}) \geq d_{\text{safe}}, \quad t = 0, \dots, T, \quad (11)$$

where ϕ_{grasp} enforces the terminal grasp condition, $q_{\min}$ and $q_{\max}$ specify the joint limits, and d_{col} denotes the minimum distance between the robot and the scene objects, constrained by the predefined safety margin d_{safe} . The objective promotes smooth joint motions, while the constraints ensure kinematic feasibility and collision avoidance. Objects admitting a feasible solution are then collected into a reachable set $\mathcal{S}_r = \{o_i \in \mathcal{S} \mid \rho(o_i) \geq \tau_r \wedge \exists q_{0:T}^*\}$, which is subsequently used for manipulability analysis.

2) *Manipulability-Aware Object Ordering*: We further rank the objects in $\mathcal{S}_r$ according to their manipulability to favor kinematically advantageous candidates for assembly. Specifically, for each $o_i \in \mathcal{S}_r$, we solve a damped least-squares inverse kinematics problem to obtain a joint configuration q_i corresponding to its desired grasp pose x_i^g . At q_i , the positional Yoshikawa manipulability [38] score is

$$m(o_i) = \sqrt{\det(J_p(q_i) J_p(q_i)^\top)}, \quad (12)$$

where $J_p(q_i)$ denotes the positional component of the Jacobian. A larger $m(o_i)$ indicates greater dexterity around the grasp configuration and thus more flexibility for subsequent motion. The objects are thus sorted in descending order of their manipulability scores, yielding a ranked set $\hat{\mathcal{S}}_r = \{o_{(1)}, o_{(2)}, \dots, o_{(N_r)}\}$, where $m(o_{(1)}) \geq m(o_{(2)}) \geq \dots \geq m(o_{(N_r)})$ and $N_r = |\mathcal{S}_r|$. This ordering provides a kinematically informed object preference for the subsequent LGP-based motion planning.

C. LGP-Based Motion Planning and Execution

Given the high-level task plan $\mathcal{P}$ and the manipulability-ranked object set $\hat{\mathcal{S}}_r$, we then generate executable robot motions to accomplish the desired assembly task.

1) *Symbolic Action Plan Determination*: For each short-horizon task plan $\mathcal{P}_k$, we first associate its graph-level components with concrete object instances in the current scene to obtain a grounded symbolic action plan. Specifically, for each node $v_j \in \mathcal{V}_k$ contained in $\mathcal{P}_k$, we select the highest-ranked unused object in $\hat{\mathcal{S}}_r$ with the same object type as

$$o_j^* = \arg \max_{o_i \in \hat{\mathcal{S}}_r^{\text{ava}}} m(o_i), \quad \text{s.t. } c(o_i) = c(v_j), \quad (13)$$

where $\hat{\mathcal{S}}_r^{\text{ava}} \subseteq \hat{\mathcal{S}}_r$ denotes the currently available objects and $c(\cdot)$ represents the object type. Once selected, o_j^* is removed from $\hat{\mathcal{S}}_r^{\text{ava}}$ to avoid repeated assignment. This converts the abstract nodes in $\mathcal{P}_k$ into concrete scene objects while retaining the manipulability-based preference.

Based on this assignment, we construct a symbolic action plan for LGP. Each assembly relation $e_{i,j} \in \mathcal{E}_k$ between

nodes v_i and v_j is instantiated using their assigned objects o_i^* and o_j^* , and is represented by a grasp-and-place sequence

$$a_{2l-1} = \text{grasp}(o_i^*), \quad a_{2l} = \text{place}(o_j^*, o_i^*, r(e_{i,j})), \quad (14)$$

where l indexes the assembly relation. Following LGP formulation [9], each action induces a symbolic state transition

$$s_n = \text{succ}(s_{n-1}, a_n), \quad n = 1, \dots, 2L_k, \quad (15)$$

where s_n denotes the symbolic state, $\text{succ}(\cdot)$ is the logical successor function, and $L_k = |\mathcal{E}_k|$. The resulting symbolic action plan is

$$\pi_k^{\text{sym}} = (s_0, a_1, s_1, \dots, a_{2L_k}, s_{2L_k}), \quad (16)$$

where each action satisfies the logical preconditions of its preceding state. This plan specifies the interaction modes and associated geometric constraints for subsequent motion optimization.

2) *LGP with Active Constraints*: We then employ LGP to optimize concrete robot motions for each short-horizon task plan $\mathcal{P}_k$ by conditioning it on the symbolic action plan π_k^{sym}

$$\min_{x(\cdot), t_{1:2L_k}} \int_0^T c(x(t), \dot{x}(t), \ddot{x}(t)) dt + \psi(x(T)), \quad (17)$$

$$\text{s.t. } \forall_{n=1}^{2L_k} h_{\text{switch}}(x(t_n) | a_n, s_{n-1}) = 0, \quad (18)$$

$$\forall_{n=1}^{2L_k} g_{\text{switch}}(x(t_n) | a_n, s_{n-1}) \leq 0, \quad (19)$$

$$\forall t \in [0, T] h_{\text{path}}(x(t), \dot{x}(t) | s_{n(t)}) = 0, \quad (20)$$

$$\forall t \in [0, T] g_{\text{path}}(x(t), \dot{x}(t) | s_{n(t)}) \leq 0. \quad (21)$$

Here, $x(t)$ denotes the continuous world configuration, including the robot configuration and poses of manipulated objects. T is the total execution time of $\mathcal{P}_k$, and t_n is the switching time associated with action a_n . The cost $c(\cdot)$ penalizes undesirable motion, such as large velocities and accelerations, while $\psi(\cdot)$ specifies the terminal objective. The switch constraints h_{switch} and g_{switch} enforce geometric conditions associated with grasp and place actions, whereas h_{path} and g_{path} impose state-dependent constraints, including joint limits and collision avoidance. The active symbolic state at time t is indexed by $n(t) = \max\{k : t_n \leq t\}$.

LGP then solves the trajectory optimization (17)-(21) using KOMO [9]. Specifically, the trajectory associated with $\mathcal{P}_k$ is discretized into a sequence of waypoints $X_{\mathcal{P}_k} = \{x_{\mathcal{P}_k}^0, \dots, x_{\mathcal{P}_k}^H\}$, over which the motion objectives and constraints are jointly evaluated. Each action a_n corresponds to a subset of consecutive waypoints $\mathcal{W}_{\mathcal{P}_k, n} = \{h_{n-1}, \dots, h_n\}$, allowing geometric and collision constraints to be examined locally along each motion segment.

However, verifying collision avoidance against all objects in scene $\mathcal{S}$ at every waypoint will result in a large number of unnecessary constraints. To reduce the computational cost, we introduce an active-constraint strategy that considers only nearby objects for each motion segment. An object $o_i \in \mathcal{S}$ is included in the active set $\mathcal{O}_{\mathcal{P}_k, n}^{\text{act}}$ for action a_n if its minimum distance to the corresponding waypoints is below an activation threshold d_{act}

$$\mathcal{O}_{\mathcal{P}_k, n}^{\text{act}} = \left\{ o_i \in \mathcal{S} \mid \min_{h \in \mathcal{W}_{\mathcal{P}_k, n}} d(x_h^h, o_i) \leq d_{\text{act}} \right\}. \quad (22)$$

Collision constraints in g_{path} are then imposed only for objects in $\mathcal{O}_{\mathcal{P}_k, n}^{\text{act}}$. The active set is updated for each action, allowing different motion segments to consider different nearby objects. The resulting optimized trajectory $x(t)$ is subsequently used for robot execution.

3) *Motion Execution with Impedance Controller*: For motion execution, we convert the optimized trajectory $x(t)$ into the corresponding robot joint-space reference trajectory $q_d(t)$, together with the gripper commands required by the symbolic actions. The robot then tracks $q_d(t)$ using a joint-space impedance controller [39]

$$\tau = K_p(q_d - q) + K_d(\dot{q}_d - \dot{q}) + \tau_{\text{com}}, \quad (23)$$

where q and $\dot{q}$ denote the current joint position and velocity, K_p and K_d are the stiffness and damping matrices, and τ_{com} compensates for gravity, inertia, and friction.

After completing the current task plan $\mathcal{P}_k$, the scene $\mathcal{S}$ is updated to reflect the resulting object configurations before proceeding to the next short-horizon task plan $\mathcal{P}_{k+1}$. The LGP optimization and motion execution are then repeated sequentially until all short-horizon plans are executed and thus the desired long-horizon assembly is achieved.

V. RESULTS

In this section, we evaluate the proposed framework through comprehensive simulation and real-world experiments, together with ablation studies.

A. Experimental Setup

1) *Tasks*: We consider two representative robotic assembly tasks: **Block Stacking (BS)** and **Functional Manipulation Benchmark (FMB)** [36] (see Fig. 2). In BS, blocks of different shapes are assembled into a specified target structure, primarily evaluating long-horizon task reasoning and planning. In FMB, block components must be precisely inserted into corresponding slots on a fixed base, resembling peg-in-hole assembly and additionally requiring accurate motion execution. For BS, we vary the number of required components from 4 to 8, while FMB involves 3 to 5 components. Two scene conditions are considered: (1) **No Redundancy (NR)**, where the scene contains exactly the number of objects required for assembly; and (2) **With Redundancy (WR)**, where an equal number of random irrelevant objects are additionally placed in the scene.

2) *Baselines*: We compare the proposed framework with three baseline methods: (1) **Direct VLM (D-VLM)**: a VLM directly generates assembly action waypoints from the target image without constructing an assembly graph or using a motion planner; (2) **Full Graph LGP (FG-LGP)**: the generated assembly graph is directly provided to LGP without graph decomposition, resulting in a single long-horizon problem. (3) **VLM-MSGraph** [40]: a recent approach that extracts a structured object-relation graph from visual input and executes the assembly using a learning-based motion planner.

TABLE I

VLM GRAPH-GENERATION ACCURACY BY BENCHMARK AND MAGNITUDE, ON SCENARIOS WITHOUT DUPLICATE OBJECTS. SELF-REFERENTIAL MAJORITY-VOTE REFERENCE, N=5 CASES PER MAGNITUDE. *w/o Validator*: RETRY LOOP IS API-ERROR-ONLY. *Ours*: FULL PIPELINE WITH THE RULE-BASED GRAPH VALIDATOR ENABLED, CORRECTING THE FOUR TRIALS IT REJECTS (SECTION V-C.2). VLM-MSGGRAPH [40]: MANUALLY SCORED AGAINST THE SOURCE IMAGE, N=10 TRIALS PER MAGNITUDE.

Magnitude	Ours	w/o Validator	VLM-MSGraph
<i>Cube Stacking</i>			
4 cubes	100.0%	100.0%	100.0%
5 cubes	100.0%	100.0%	90.0%
6 cubes	100.0%	100.0%	90.0%
7 cubes	100.0%	98.0%	90.0%
8 cubes	94.0%	92.0%	90.0%
Overall	98.8%	98.0%	92.0%
<i>FMB</i>			
3 objects	94.0%	94.0%	80.0%
4 objects	100.0%	100.0%	50.0%
5 objects	92.0%	88.0%	10.0%
Overall	95.3%	94.0%	46.7%

3) *Metrics*: We primarily evaluate assembly performance using task success rate. For each task and component count, we consider 5 different target structures, each examined using 10 randomly initialized scenes $\mathcal{S}$, and the overall success rate is reported. Additionally, we measure the computational time of each framework component.

4) *Parameters and Hardware*: The following parameters are used in our experiments: ... All simulation experiments are conducted in MuJoCo on a local PC equipped with an xxx CPU and an NVIDIA GeForce xxxx GPU with xxx GB memory. Each planning task is capped at 16 GB of memory. The real-world experiments are performed using a Franka Emika Panda robotic arm controlled by the same PC.

ZZ: give details of hardware platform and parameters, which VLM is used?

B. Assembly Performance

1) *Assembly Graph Generation*: We first examine the accuracy of generating the assembly graph $\mathcal{G}$ from the target image $\mathcal{I}$. As FG-LGP use the same assembly graph generation as the proposed method, we only compare our proposed method with directly using VLM for generating the complete graph and with VLM-MSGraph. The results obtained

Asking the VLM to output pick-and-place waypoints directly from the target image, without any intermediate representation, fails outright: the returned sequences are syntactically valid but geometrically inconsistent – grasp points miss the target objects, waypoints intersect the table or previously placed parts, and placement heights violate the support order. No trial completes an assembly, so we report no success-rate table for this baseline; the failure confirms that current VLMs cannot be trusted with exact, metrically precise motion generation (Section IV-A).

FMB structures frequently include bridging relations, where one object spans multiple supporters and is partially occluded in any single view of the final assembly. Table III compares two ways of presenting such structures to the VLM, on two representative bridging scenes: a single composite image of the final assembly, which requires inferring occluded support from color fragmentation, versus a sequence of one image per build step. Sequential input reaches 100% on both scenes; single-image input drops to 80–90%, with errors concentrated on the occluded bridging relations. We adopt sequential, per-step image input for all FMB experiments in this work.

Table I compares our method’s graph extraction against VLM-MSGraph’s semantic layer, both scored on scenarios without duplicate objects, as VLM-MSGraph’s open-vocabulary matching provides no mechanism for disambiguating them. On cube stacking, a planar layout, both methods are close: 98.8% for ours against 92.0% for VLM-MSGraph. On FMB, where objects must be related across insertion layers under a strict support order, the gap widens sharply – 95.3% against 46.7%, falling to 10.0% at five objects.

ZZ: here give results: about using different VLM in graph extraction accuracy, try to put everything in one table since we are lacking of space

2) *Task Planning and Motion Execution*: Before comparing decomposition strategies, we first check whether decomposition is needed at all. Solving the smallest FMB configuration (three objects) as a single monolithic LGP problem – without Smart’s active-constraint batching or Global’s fixed, layer-based split – succeeds in 9 of 50 trials without redundancy and in none of the 50 redundancy trials (Combined columns of Table IV); every failure exhausts the 16 GB memory cap rather than the time budget. Some form of decomposition is thus a precondition for solving these assemblies at all; the remainder of this section compares two decomposition strategies against this monolithic baseline.

VLM-MSGraph’s perception stage grounds each graph node into the scene through open-vocabulary detection and segmentation; we do not re-implement or evaluate this stage. Instead, both methods receive an identical, externally supplied task assignment – object, source pose, target pose, and order – taken from our own method’s output, isolating the trajectory-generation step. Its executor performs no collision checking and no motion planning: each pick and place is a direct interpolation from the current pose to the target pose. This keeps a modest success rate on the smallest, least cluttered configurations, but degrades quickly as the object count grows and as the required motions become less permissive: cube-stacking success falls from 16.0% at four cubes to at or near 0% from six cubes onward, and on FMB, where insertions require precise approach and support-order compliance, no trial succeeds at any object count (Table IV, rightmost columns). Even given a correct task assignment, an executor without collision-aware planning is not viable beyond the simplest cases.

Table IV and Table V compare Global’s fixed, layer-

based decomposition against Smart’s VLM-driven, active-constraint decomposition. Smart solves every cube-stacking configuration at 100%; Global degrades with scale, most severely under redundancy, falling to 56–58% at five to six cubes, 38% at seven, and 0% at eight. On FMB, Smart’s pooled success rate is 97.0% against Global’s 31.3%, and Global collapses entirely at five objects (0.0% under both conditions), where the solver must disambiguate distractors while exhaustively checking collisions within an already large batch. Solving time follows the same pattern: Smart grows slowly with size, while Global grows sharply under redundancy – from 175 s at six cubes to 399 s at seven, and to 284.6 s at four FMB objects under redundancy against Smart’s 223.5 s, with no successful Global trial left to time at five objects. The monolithic Combined variant, which merges every object’s goal into a single LGP terminal instead of decomposing the task, fares far worse: it succeeds only on the easiest FMB configuration without redundancy (18.0%) and fails at every other cell tested, including cube stacking at four cubes (the only magnitude executed under Combined; five to eight cubes were not run, as a strictly easier magnitude already failing completely implies the harder ones do too) – decomposition itself, not merely the choice between Smart and Global, is what makes these problems tractable.

ZZ: here give results: about the motion execution success rates, in terms of using LGP things, put things in one table

3) *Real-World Experiments*: We validate the pipeline on the physical robot. Representative cube-stacking and FMB instances are planned by Smart, and the resulting trajectories are parsed into joint-space motions and gripper events and executed on the Franka Panda as described in Section IV-C. Figures 5 and 6 show the initial scene and execution sequence for each magnitude.

ZZ: give figures of real-world experiments

C. Ablation Studies

1) *VLM for Graph Generation*: Table II compares four candidate VLMs on FMB graph extraction. Gemini-3-Flash and Qwen3.6-Plus are the only models to reach 100% on both test batches; Qwen3.5-Plus and Qwen3.5-Flash trail substantially. Given comparable accuracy between the top two, we adopt Gemini-3-Flash as the VLM front end for the remainder of this work.

2) *Assembly Graph Generation Validator*: During the graph-generation evaluation (Table I), the validator is triggered in four of the 400 trials, each in a different case. All four violations are structural: a bridging object that omits a supporter standing between its other supporters at the same height, a bridging object whose supporters lie on different layers, and a support edge without a valid supporter index. Without the validator, the rejected graph is passed downstream as generated, and each of the four cases reaches 90% accuracy. With the validator, the violated rule is returned to the VLM as feedback and the graph is regenerated. We sample this regeneration ten times per case, matching the ten trials per case of the main evaluation; all 40 regenerated graphs match the reference (Wilson 95% CI

TABLE II
SMALL-SCALE VLM BACKBONE SELECTION EXPERIMENT COMPARING CANDIDATE MODELS ON FMB GRAPH-GENERATION ACCURACY, N=10 TRIALS PER MODEL-TEST CELL; MOTIVATES THE CHOICE OF GEMINI-3-FLASH AS THE VLM FRONT-END USED THROUGHOUT THE REST OF THIS SECTION.

Model	Test-1	Test-3
Gemini-3-Flash	100.0% (10/10)	100.0% (10/10)
Qwen3.6-Plus	100.0% (10/10)	100.0% (10/10)
Qwen3.5-Plus	70.0% (7/10)	60.0% (6/10)
Qwen3.5-Flash	0.0% (0/10)	0.0% (0/10)

TABLE III
FMB SUPPORT-GRAPH ACCURACY FOR GEMINI-3-FLASH UNDER TWO IMAGE-INPUT FORMATS, ON TWO REPRESENTATIVE BRIDGING-RELATION SCENES, N=10 TRIALS PER CELL.

	Single-Image	Sequential-Image
Test 6	80.0% (8/10)	100.0% (10/10)
Test 7	90.0% (9/10)	100.0% (10/10)

[91.2%, 100%]), raising each case to 100%. Graphs that pass the validator never enter the retry loop, so the remaining trials are unaffected, and overall accuracy rises from 98.0% to 98.8% on cube stacking and from 94.0% to 95.3% on FMB.

The validator checks only the internal consistency of the predicted graph and uses neither the target image nor the part list. Errors that leave the graph self-consistent, such as a wrong position label, a misidentified object type, or a missing object, therefore pass it; this is why the three lowest-scoring cases (70%) are not among the four the validator catches.

3) *Reachability-Manipulability-Aware Object Ordering*: **ZZ: here: compare the influence of reachability and manipulability, design some cases to show that undesirable object grasping is avoided. better give some metrics/numbers to show the influence, like how many percentages/trails that we prevent that**

4) *Active Constraints for LGP*: **ZZ: here: compare with and without active constraints (your previous global LGP), in terms of computational time, show that active constraints indeed accelerate things**

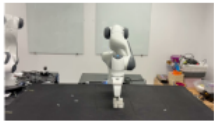
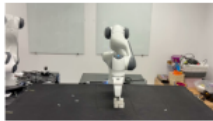
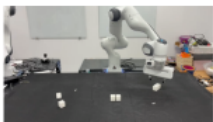
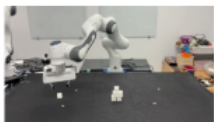
	Initial Scene	Robot Execution				
4 Cubes 						
5 Cubes 						
6 Cubes 						
7 Cubes 						
8 Cubes 						

Fig. 5. Representative real-robot execution: initial scene and execution-sequence photos for cube-stacking assemblies of 4 to 8 cubes on the physical Franka Panda platform.

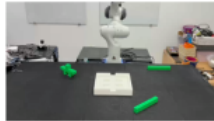
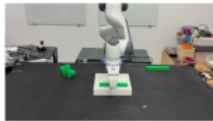
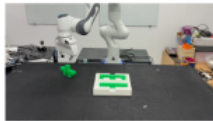
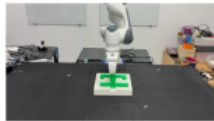
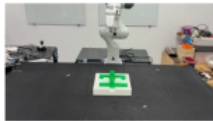
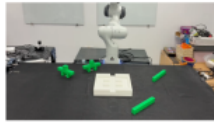
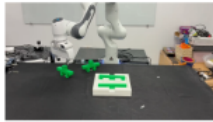
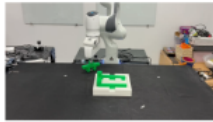
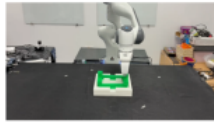
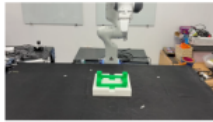
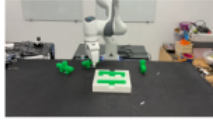
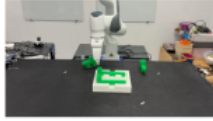
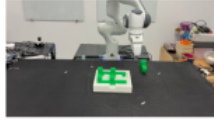
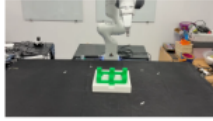
	Initial Scene	Robot Execution				
3 Objects 						
4 Objects 						
5 Objects 						

Fig. 6. Representative real-robot execution: initial scene and execution-sequence photos for FMB assemblies of 3 to 5 objects on the physical Franka Panda platform.

TABLE IV

SUCCESS RATE (%) BY BENCHMARK, MAGNITUDE, AND REDUNDANCY CONDITION. SMART, GLOBAL, AND COMBINED SUCCEED VIA CONSTRAINED LGP PLANNING (COMBINED MERGES EVERY OBJECT’S GOAL INTO A SINGLE MONOLITHIC TERMINAL INSTEAD OF DECOMPOSING THE TASK); VLM-MSGGRAPH [40] IS EVALUATED BY WHETHER ITS NAIVE INTERPOLATION EXECUTES WITHOUT COLLISION, GIVEN THE SAME SCENES AND TASK ASSIGNMENT. CUBE STACKING: N=1000 (SMART/GLOBAL), N=500 (VLM-MSGGRAPH), N=100 AT FOUR CUBES / NOT EXECUTED AT FIVE–EIGHT CUBES (COMBINED, SEE TEXT). FMB: N=50 PER CELL (SMART/GLOBAL/COMBINED, NO EXCLUSIONS – RE-RUN WITH NO WALL-CLOCK TIMEOUT, 16GB MEMORY CAP ONLY), N=300 (VLM-MSGGRAPH).

	Smart (Ours)		Global		Combined		VLM-MSGGraph	
	NR	R	NR	R	NR	R	NR	R
<i>Cube Stacking</i>								
4 cubes	100.0	100.0	100.0	78.0	0.0	0.0	16.0	8.0
5 cubes	100.0	100.0	96.7	56.0	0.0	0.0	2.0	4.0
6 cubes	100.0	100.0	100.0	58.0	0.0	0.0	0.0	0.0
7 cubes	100.0	100.0	93.3	38.0	0.0	0.0	0.0	0.0
8 cubes	100.0	100.0	60.0	0.0	0.0	0.0	4.0	0.0
<i>FMB</i>								
3 objects	96.0	96.0	92.0	62.0	18.0	0.0	0.0	0.0
4 objects	98.0	100.0	22.0	12.0	0.0	0.0	0.0	0.0
5 objects	96.0	96.0	0.0	0.0	0.0	0.0	0.0	0.0

TABLE V

AVERAGE SOLVING TIME IN SECONDS OVER SUCCESSFUL TRIALS. CUBE STACKING GLOBAL-R AT EIGHT CUBES HAS NO SUCCESSFUL TRIAL; GLOBAL HAS NO SUCCESSFUL FMB TRIAL AT FIVE OBJECTS (EITHER CONDITION); COMBINED HAS NO SUCCESSFUL TRIAL AT ANY CUBE-STACKING MAGNITUDE, AND NONE ON FMB EXCEPT 3 OBJECTS (NR).

	Smart (Ours)		Global		Combined	
	NR	R	NR	R	NR	R
<i>Cube Stacking</i>						
4 cubes	28.7	23.3	26.9	31.8	—	—
5 cubes	31.2	34.2	38.1	63.7	—	—
6 cubes	36.8	42.8	37.2	174.8	—	—
7 cubes	40.9	57.7	69.0	399.2	—	—
8 cubes	55.0	61.6	77.9	—	—	—
<i>FMB</i>						
3 objects	209.3	209.5	282.2	215.5	247.5	—
4 objects	229.3	223.5	255.2	284.6	—	—
5 objects	274.7	285.5	—	—	—	—

TABLE VI

PER-STAGE COMPUTATION TIME (MS). REACHABILITY FILTERING COMPRISES THE GEOMETRIC ACCESSIBILITY SCORE (STAGE 1) AND THE KOMO FEASIBILITY CHECK (STAGE 2). MEANS OVER 10 SCENES PER CELL.

	Reachability				Manipulability	
	Stage 1: score		Stage 2: KOMO		NR	R
	NR	R	NR	R		
<i>Cube Stacking</i>						
4 cubes	0.13	0.17	963.5	2042.4	10.9	20.7
5 cubes	0.13	0.18	1232.2	2649.2	13.3	25.5
6 cubes	0.16	0.23	1499.0	3246.1	15.8	31.3
7 cubes	0.16	0.24	1802.4	3883.1	18.7	35.1
8 cubes	0.18	0.29	2076.1	4439.9	21.4	39.0
<i>FMB</i>						
3 objects	0.13	0.17	785.1	1613.1	8.7	18.2
4 objects	0.15	0.20	1042.0	2203.4	11.5	23.7
5 objects	0.16	0.23	1336.5	2748.7	14.9	28.3

VI. DISCUSSION

A. Role of VLM

VII. CONCLUSION

REFERENCES

- [1] R. Li and H. Qiao, "A survey of methods and strategies for high-precision robotic grasping and assembly tasks—some new trends," *IEEE/ASME Transactions on Mechatronics*, vol. 24, no. 6, pp. 2718–2732, 2019.
- [2] M. Daneshmand, F. Noroozi, C. Corneanu, F. Mafakheri, and P. Fiorini, "Industry 4.0 and prospects of circular economy: a survey of robotic assembly and disassembly," *The International Journal of Advanced Manufacturing Technology*, vol. 124, no. 9, pp. 2973–3000, 2023.
- [3] Y. Jiang, Z. Huang, B. Yang, and W. Yang, "A review of robotic assembly strategies for the full operation procedure: planning, execution and evaluation," *Robotics and Computer-Integrated Manufacturing*, vol. 78, p. 102366, 2022.
- [4] M. Faccio, M. Bottin, and G. Rosati, "Collaborative and traditional robotic assembly: a comparison model," *The International Journal of Advanced Manufacturing Technology*, vol. 102, no. 5, pp. 1355–1372, 2019.
- [5] Z. Zhao, S. Cheng, Y. Ding, Z. Zhou, S. Zhang, D. Xu, and Y. Zhao, "A survey of optimization-based task and motion planning: From classical to learning approaches," *IEEE/ASME Transactions On Mechatronics*, vol. 30, no. 4, pp. 2799–2825, 2024.
- [6] H. Guo, F. Wu, Y. Qin, R. Li, K. Li, and K. Li, "Recent trends in task and motion planning for robotics: A survey," *ACM Computing Surveys*, vol. 55, no. 13s, pp. 1–36, 2023.
- [7] C. Aeronautiques, A. Howe, C. Knoblock, I. D. McDermott, A. Ram, M. Veloso, D. Weld, D. W. Sri, A. Barrett, D. Christianson, *et al.*, "Pddl—the planning domain definition language," *Technical Report, Tech. Rep.*, 1998.
- [8] Y.-q. Jiang, S.-q. Zhang, P. Khandelwal, and P. Stone, "Task planning in robotics: an empirical comparison of pddl-and asp-based systems," *Frontiers of Information Technology & Electronic Engineering*, vol. 20, no. 3, pp. 363–373, 2019.
- [9] M. Toussaint, "Logic-geometric programming: an optimization-based approach to combined task and motion planning," in *Proceedings of the 24th International Conference on Artificial Intelligence*. AAAI Press, 2015, p. 1930–1936.
- [10] M. Toussaint and M. Lopes, "Multi-bound tree search for logic-geometric programming in cooperative manipulation domains," in *2017 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2017, pp. 4044–4051.
- [11] C. Gkournelos, C. Konstantinou, and S. Makris, "An llm-based approach for enabling seamless human-robot collaboration in assembly," *CIRP Annals*, vol. 73, no. 1, pp. 9–12, 2024.
- [12] J. Ao, F. Wu, Y. Wu, A. Swiki, and S. Haddadin, "Llm-as-bt-planner: Leveraging llms for behavior tree generation in robot task planning," in *2025 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2025, pp. 1233–1239.
- [13] Z. Zhou, J. Song, K. Yao, Z. Shu, and L. Ma, "Isr-llm: Iterative self-refined large language model for long-horizon sequential task planning," in *2024 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2024, pp. 2081–2088.
- [14] J. Wang, E. Shi, H. Hu, C. Ma, Y. Liu, X. Wang, Y. Yao, X. Liu, B. Ge, and S. Zhang, "Large language models for robotics: Opportunities, challenges, and perspectives," *Journal of Automation and Intelligence*, vol. 4, no. 1, pp. 52–64, 2025.
- [15] J. Qi, L. Lu, F. Wang, H.-Y. Lee, D. Navarro-Alarcon, Z. Zhang, and P. Zhou, "Llm-driven symbolic planning and hierarchical imitation learning for long-horizon deformable object assembly," *Robotics and Computer-Integrated Manufacturing*, vol. 97, p. 103096, 2026.
- [16] V. N. Hartmann, A. Orthey, D. Driess, O. S. Oguz, and M. Toussaint, "Long-horizon multi-robot rearrangement planning for construction assembly," *IEEE Transactions on Robotics*, vol. 39, no. 1, pp. 239–252, 2023.
- [17] A. B. Gunji, B. B. V. L. Deepak, C. M. V. A. R. Bahubalendruni, and D. B. B. Biswal, "An optimal robotic assembly sequence planning by assembly subsets detection method using teaching learning-based optimization algorithm," *IEEE Transactions on Automation Science and Engineering*, vol. 15, no. 3, pp. 1369–1385, 2018.
- [18] P. Tang and J. Xiao, "Automatic generation of a high-level contact state graph for assembly between curved objects," in *2007 IEEE International Symposium on Assembly and Manufacturing*, 2007, pp. 197–202.
- [19] H. Chen, W. Wan, K. Koyama, and K. Harada, "Planning to build block structures with unstable intermediate states using two manipulators," *IEEE Transactions on Automation Science and Engineering*, vol. 19, no. 4, pp. 3777–3793, 2022.
- [20] Z. Wang, Q. Pan, Y. Zhou, R. Jiang, J. Ma, B. He, and X. Li, "Long-sequence task planning for flexible assembly with spatio-temporal scene graph," *IEEE Transactions on Automation Science and Engineering*, vol. 22, pp. 23 594–23 606, 2025.
- [21] M. Toussaint, "Logic-geometric programming: an optimization-based approach to combined task and motion planning," in *Proceedings of the 24th International Conference on Artificial Intelligence*, ser. IJCAI'15. AAAI Press, 2015, p. 1930–1936.
- [22] A. T. Le, P. Kratzer, S. Hagenmayer, M. Toussaint, and J. Mainprice, "Hierarchical human-motion prediction and logic-geometric programming for minimal interference human-robot tasks," in *2021 30th IEEE International Conference on Robot and Human Interactive Communication (RO-MAN)*, 2021, pp. 7–14.
- [23] C. V. Braun, J. Ortiz-Haro, M. Toussaint, and O. S. Oguz, "Rhh-llp: Receding horizon and heuristics-based logic-geometric programming for task and motion planning," in *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2022, pp. 13 761–13 768.
- [24] T. Xue, A. Razmjoo, and S. Calinon, "D-llp: Dynamic logic-geometric program for reactive task and motion planning," in *2024 IEEE International Conference on Robotics and Automation (ICRA)*, 2024, pp. 14 888–14 894.
- [25] K. T. Ly, V. Semenov, M. Risiglione, W. Merkt, and I. Havoutis, "R-llp: A reachability-guided logic-geometric programming framework for optimal task and motion planning on mobile manipulators," in *2024 IEEE International Conference on Robotics and Automation (ICRA)*, 2024, pp. 14 917–14 923.
- [26] Y. Ma, Z. Song, Y. Zhuang, J. Hao, and I. King, "A survey on vision-language-action models for embodied ai," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 37, no. 7, pp. 3031–3051, 2026.
- [27] X. Li, G. Tian, and Y. Cui, "Semantically guided task planning: Supervised vision-language-action model by large language models," *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 36, no. 5, pp. 7411–7425, 2026.
- [28] Y. Han, K. Wu, Q. Shao, R. Xiao, Z. Wang, C. Jiang, Y. Xiao, L. Hu, and Y. Lou, "AppleVlm: End-to-end autonomous driving with advanced perception and planning-enhanced vision-language models," *IEEE Transactions on Intelligent Transportation Systems*, vol. 27, no. 6, pp. 7028–7043, 2026.
- [29] H. Tian, K. Reddy, Y. Feng, M. Qudus, Y. Demiris, and P. Angeloudis, "Large (vision) language models for autonomous vehicles: Current trends and future directions," *IEEE Transactions on Intelligent Transportation Systems*, vol. 27, no. 1, pp. 187–210, 2026.
- [30] G. Chen, Q. Yang, J. Li, C. L. P. Chen, and C. Yang, "Cooperative control framework for dual-arm robot enhanced by vision language model and reinforcement learning," *IEEE Transactions on Automation Science and Engineering*, vol. 23, pp. 4314–4328, 2026.
- [31] P. Hao, S. Cui, J. Wei, T. Lu, Y. Cai, and S. Wang, "Learn-gen-plan: Bridging the gap between vision language models and real-world long-horizon dexterous manipulations," *IEEE Transactions on Automation Science and Engineering*, vol. 22, pp. 15 638–15 649, 2025.
- [32] R. Römer, Y. Zhang, Y. Li, and A. P. Schoellig, "Clare: Continual learning for vision-language-action models via autonomous adapter routing and expansion," *IEEE Robotics and Automation Letters*, vol. 11, no. 7, pp. 8331–8338, 2026.
- [33] J. Wen, Y. Zhu, J. Li, M. Zhu, Z. Tang, K. Wu, Z. Xu, N. Liu, R. Cheng, C. Shen, Y. Peng, F. Feng, and J. Tang, "Tinyvla: Toward fast, data-efficient vision-language-action models for robotic manipulation," *IEEE Robotics and Automation Letters*, vol. 10, no. 4, pp. 3988–3995, 2025.
- [34] Z. Su, W. Kong, H. Dong, and H. Dong, "Ig-rft: An interaction-guided rl framework for vla models in long-horizon robotic manipulation," *IEEE Transactions on Automation Science and Engineering*, vol. 23, pp. 14 131–14 146, 2026.
- [35] Y. Yang, Z. Li, and Z. Hu, "Vision-language-action model-based event-triggered admittance control of a mobile manipulator for power

- substation live-maintaining,” *IEEE Transactions on Fuzzy Systems*, vol. 34, no. 3, pp. 881–895, 2026.
- [36] J. Luo, C. Xu, F. Liu, L. Tan, Z. Lin, J. Wu, P. Abbeel, and S. Levine, “Fmb: A functional manipulation benchmark for generalizable robotic learning,” *The International Journal of Robotics Research*, vol. 44, no. 4, pp. 592–606, 2025.
- [37] M. Toussaint, “Newton methods for k-order markov constrained motion problems,” *arXiv preprint arXiv:1407.0414*, 2014.
- [38] T. Yoshikawa, “Manipulability of robotic mechanisms,” *The International Journal of Robotics Research*, vol. 4, no. 2, pp. 3–9, 1985.
- [39] C. Ott, A. Albu-Schaffer, A. Kugi, and G. Hirzinger, “On the passivity-based impedance control of flexible joint robots,” *IEEE Transactions on Robotics*, vol. 24, no. 2, pp. 416–429, 2008.
- [40] S. Li, Z. Yan, Z. Wang, and Y. Gao, “Vlm-msgraph: Vision language model-enabled multi-hierarchical scene graph for robotic assembly,” *Robotics and Computer-Integrated Manufacturing*, vol. 94, p. 102978, 2025.

ZZ: video content: (1) Experimental setup needs to show an overview of all five target structures for all object numbers and for both tasks (BS:(4-8) $\times$ 5 = 25 structures, FMB:(3-5) $\times$ 5 = 15 structures). (2) prompt for VLM